

WORKSHEET MAHASISWA

Modul 2 — Vanilla JavaScript dan Interaktivitas Web

Dasar Java Script · DOM dan Event · Validasi Form · Asynchronous · Debugging

Mata Kuliah Proyek 3 — Proyek Pengembangan Perangkat Lunak Berbasis Web · D3 Teknik Informatika

Identitas	Isian
Nama	Putra Suyapratama
NIM	251511056
Kelas	D3-2B
Tanggal Mulai	14 September 2026
Tanggal Pengumpulan	18 September 2026
URL Repository	https://github.com/hmzahigball/proyek3-modul2.git

Petunjuk Penggunaan

- ☑ Isi worksheet bersamaan dengan pengerjaan modul, bukan setelah seluruh tugas selesai.
- ☑ Pada eksperimen, tulis prediksi sebelum menjalankan kode, lalu catat hasil aktual dan bukti.
- ☑ Gunakan Console, Elements, Network, dan tampilan halaman sebagai sumber bukti sesuai aktivitas.
- ☑ Gunakan hint secara bertahap; catat percobaan sebelum membuka hint berikutnya.
- ☑ Catat penggunaan AI atau sumber bantuan secara transparan dan lakukan verifikasi mandiri.
- ☑ Worksheet dikumpulkan dalam PDF bersama bukti yang disyaratkan pada tugas.

Alur Belajar Alur pembelajaran dilaksanakan melalui lima tahap, yaitu membuat prediksi, menjalankan percobaan, mengamati hasil, menjelaskan temuan, dan melakukan perbaikan. Bukti proses memiliki kedudukan yang sama pentingnya dengan hasil akhir.

A. Eksperimen

A1. Eksperimen Console dan Tipe Data

Isi kolom *Prediksi* sebelum membuka *Console*. Setelah menjalankan kode, catat hasil aktual dan alasan perbedaannya.

Ekspresi atau Pemeriksaan	Prediksi	Hasil Aktual	Tipe Data atau Makna	Penjelasan
typeof nama	String	String	String	Menunjukkan bahwa variabel nama bertipe data teks (String) karena nilainya berupa karakter 'Rani' yang diapit tanda petik.
typeof nilaiTeks	String	String	String	Menunjukkan bahwa variabel nilaiTeks bertipe data teks (String) karena nilainya '80' diapit tanda petik, meskipun isinya berupa karakter angka.
typeof nilaiAngka	Number	Number	Number	Menunjukkan bahwa variabel nilaiAngka bertipe numerik (Number) karena ditulis langsung angka 80 tanpa tanda petik.
nilaiTeks + 5	805	805	String	Karena salah satu operand bertipe String ('80'), operator + berfungsi sebagai penggabung sehingga menghasilkan string '805'.
Number(nilaiTeks) + 5	85	85	Number	Fungsi Number() mengonversi string '80' menjadi angka 80, sehingga operator + berfungsi sebagai operasi penjumlahan matematika ($80 + 5 = 85$).
nilaiTeks == nilaiAngka	True	True	Boolean	Operator perbandingan longgar membandingkan nilai dengan melakukan konversi tipe data otomatis. Karena nilainya sama setelah dikonversi ($80 == 80$), hasilnya true.
nilaiTeks === nilaiAngka	False	False	Boolean	Operator perbandingan ketat membandingkan nilai sekaligus tipe datanya tanpa adanya konversi tipe otomatis. Karena tipenya berbeda (String vs Number), maka hasilnya false.

Ekspresi atau Pemeriksaan	Prediksi	Hasil Aktual	Tipe Data atau Makna	Penjelasan
Number(teks bukan angka) Number('hello')	1	NaN	Number	Fungsi Number() tidak dapat mengonversi tipe data string, jika dilakukan maka hasilnya adalah NaN (Not a Number).

Apa bukti bahwa angka 80 dan teks "80" berbeda?

Angka 80 menunjukkan bahwa variable memiliki nilai 80 sebagai angka numerik (number), sedangkan '80' merupakan nilai bagi variable String, bukan angka numerik yang bisa dioperasikan dengan numerik lainnya.

Mengapa operator + dapat menghasilkan hasil yang berbeda?

Operator + dapat memiliki makna yang berbeda tergantung dengan tipe data dari operand. Operator + berfungsi untuk menjumlahkan nilai jika digunakan untuk tipe data Number ($80 + 5 = 85$), namun operator + dapat berfungsi sebagai penambah karakter jika digunakan untuk tipe data String ('80' + 5 = '805').

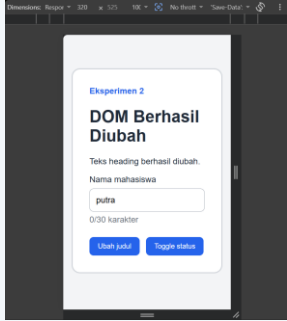
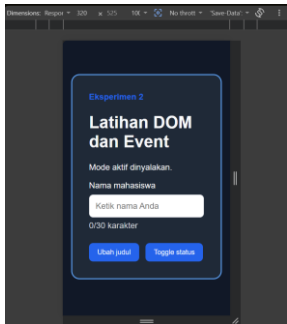
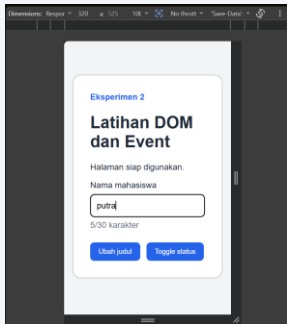
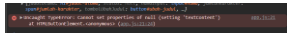
Mengapa perbandingan ketat === menjadi pilihan awal yang lebih aman?

Perbandingan ketat (===) merupakan pilihan yang lebih aman karena membandingkan nilai dan tipe datanya sekaligus tanpa melakukan konversi tipe data otomatis, berbeda dengan perbandingan (==) yang dapat menghasilkan bug akibat konversi otomatis tipe datanya. Perbandingan ketat ini membuat kode lebih eksplisit dan memastikan variable yang dibandingkan benar-benar memiliki tipe data dan nilai yang identik.

A2. Eksperimen DOM dan Event

Alur yang diamati Alur yang diamati dimulai ketika pengguna melakukan suatu aksi. Browser kemudian membangkitkan event, fungsi penanganan dijalankan, elemen DOM atau class diperbarui, dan browser merender perubahan pada tampilan.

Langkah	Prediksi sebelum diuji	Hasil aktual	Bukti dan Penjelasan
1 · Memilih elemen	#judul-utama #status #nama #jumlah-karakter #ubah-judul #toggle-status		Method document.querySelector() berhasil memetakan elemen karena seluruh ID selector (#...) di app.js cocok secara presisi dengan atribut id pada index.html. Penggunaan atribut defer pada tag <script> memastikan struktur HTML (DOM Tree) selesai dibaca browser sebelum JavaScript dijalankan.

Langkah	Prediksi sebelum diuji	Hasil aktual	Bukti dan Penjelasan
2 · Mengubah heading	“Latihan DOM dan Event” akan berubah menjadi “DOM Berhasil Diubah” dan status “Halaman siap digunakan” berubah menjadi “Teks heading berhasil diubah.”.		Teks berubah tanpa reload karena JavaScript memanipulasi langsung representasi memori DOM di sisi klien (client-side) melalui properti <code>.textContent</code> . Tidak ada permintaan (request) baru ke server; browser hanya melakukan render ulang pada elemen yang dimodifikasi.
3 · Mengubah class dan atribut aria-pressed	Ketika button diklik, maka akan mengaktifkan / menonaktifkan toggle “is-active”, dan aria-pressed menjadi true , begitu juga dengan teks status akan berubah.		Pemanggilan <code>document.body.classList.toggle('is-active')</code> secara dinamis menambah/mencabut class <code>is-active</code> . Perubahan class ini memicu aturan CSS <code>.is-active</code> yang sudah disiapkan di <code>style.css</code> . Penambahan <code>aria-pressed</code> memastikan pembaca layar (screen reader) mengetahui status tombol aktif/non-aktif.
4 · Event input dan penghitung karakter	Keterangan pada #jumlah-karakter akan berubah, menampilkan total dari karakter yang ada pada kotak input, keterangan akan berubah ketika terjadinya input dari user.		Event input terpicu secara langsung setiap kali terjadi perubahan nilai pada input form. Kode mengambil nilai panjang string melalui <code>event.target.value.length</code> dan langsung memperbarui teks pada elemen penghitung melalui <code>jumlahKarakter.textContent</code> .
5 · Selector tidak cocok dan menghasilkan nilai null	Terdapat pesan dari console yang menunjukkan bahwa #status tidak ditemukan, atau bahkan menunjukkan keterangan null pada console.		Karena selector <code>#status</code> tidak menemukan elemen yang cocok di dokumen HTML, <code>querySelector</code> mengembalikan nilai <code>null</code> . Mencoba mengakses atau mengisi properti <code>.textContent</code> pada objek <code>null</code> merupakan operasi tidak sah yang menyebabkan JavaScript melempar fatal error (<code>TypeError</code>).

Tuliskan pesan error ketika selector tidak cocok, meliputi nama file, nomor baris, dan inti pesan:

Nama file: app.js

Nomor baris: Baris 21 (kolom 22) [app.js:21:22]

Inti pesan error: Uncaught TypeError: Cannot set properties of null (setting 'textContent')

Keterangan: Error ini terjadi karena variabel status bernilai null (selector #status tidak menemukan elemen HTML yang cocok), sehingga browser gagal menyetel properti textContent pada nilai null.

Jelaskan perbedaan bukti yang diperoleh sebelum dan sesudah pemeriksaan nilai null melalui function ubahStatus.

Sebelum pemeriksaan: Console menampilkan pesan fatal error berwarna merah berupa Uncaught TypeError: Cannot set properties of null (setting 'textContent').

Sesudah pemeriksaan: Console tidak lagi mengalami crash/fatal error.

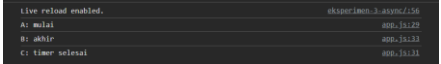
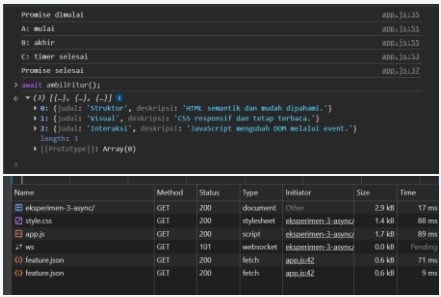
Jelaskan pembagian peran JavaScript dan CSS ketika class is-active diubah melalui mekanisme toggle.

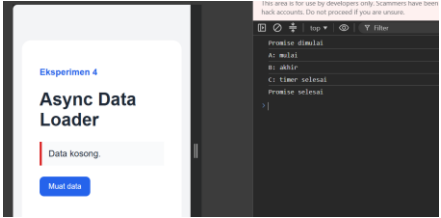
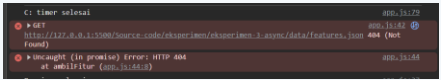
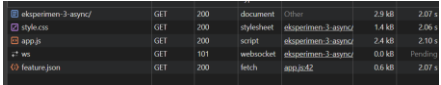
JavaScript (Logika & State): Menangani event klik pengguna untuk menambah/menghapus class is-active pada <body> serta memperbarui atribut aria-pressed. JS hanya mengatur status tanpa mengubah style warna secara langsung.

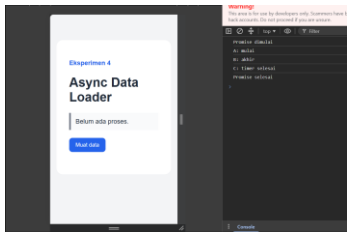
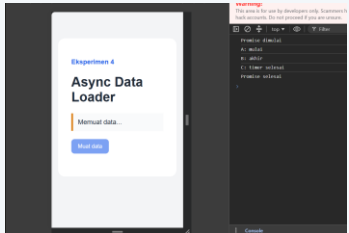
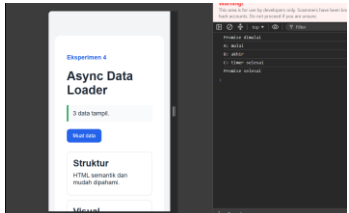
CSS (Tampilan & Presentasi): Menentukan styling visual (seperti warna latar gelap, kartu, teks, dan transisi) yang otomatis diterapkan ketika class is-active aktif.

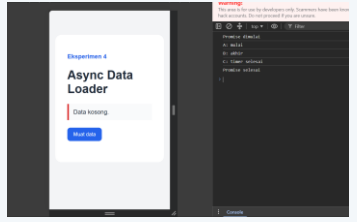
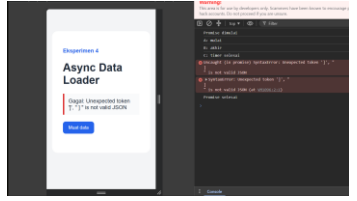
A3. Eksperimen Promise, fetch, dan State UI

Jalankan proyek melalui server pengembangan lokal. Gunakan tampilan halaman, Console, dan Network sebagai bukti pengujian.

Skenario atau Langkah	Prediksi	Hasil Aktual	Bukti dari Console, Network, atau Antarmuka
Urutan A, B, C pada setTimeout(0)	Urutannya akan A->C->B	A -> B -> C	 <pre> live reload enabled. eksperimen-3.async/156 A: mulai 10:10:29 B: akhir 10:10:33 C: timer selesai 10:10:31 </pre>
Data JSON valid	Nilai didalam JSON akan ditampilkan pada tab console, sesudah kita memanggil ambilFitur() secara manual melalui await.	Nilai dalam JSON berhasil ditampilkan pada tab console, dengan status HTTP 200.	 <pre> Promise dimulai 10:10:35 A: mulai 10:10:51 B: akhir 10:10:55 C: timer selesai 10:10:53 Promise selesai 10:10:37 > await ambilFitur(); < * (0) [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 0: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 1: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 2: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 3: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 4: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 5: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 6: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 7: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 8: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 9: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 10: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 11: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 12: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 13: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 14: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 15: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 16: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 17: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 18: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 19: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 20: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 21: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 22: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 23: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 24: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 25: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 26: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 27: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 28: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 29: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 30: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 31: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 32: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 33: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 34: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 35: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 36: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 37: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 38: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 39: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 40: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 41: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 42: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 43: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 44: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 45: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 46: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 47: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 48: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 49: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 50: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 51: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 52: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 53: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 54: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 55: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 56: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 57: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 58: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 59: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 60: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 61: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 62: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 63: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 64: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 65: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 66: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 67: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 68: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 69: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 70: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 71: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 72: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 73: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 74: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 75: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 76: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 77: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 78: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 79: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 80: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 81: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 82: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 83: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 84: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 85: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 86: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 87: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 88: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 89: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 90: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 91: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 92: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 93: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 94: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 95: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 96: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 97: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 98: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 99: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 100: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 101: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 102: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 103: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 104: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 105: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 106: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 107: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 108: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 109: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 110: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 111: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 112: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 113: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 114: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 115: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 116: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 117: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 118: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 119: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 120: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 121: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 122: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 123: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 124: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 125: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 126: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 127: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 128: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 129: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 130: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 131: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 132: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 133: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 134: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 135: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 136: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 137: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 138: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 139: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 140: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 141: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 142: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 143: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 144: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 145: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 146: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 147: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 148: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 149: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 150: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 151: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 152: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 153: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 154: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 155: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 156: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 157: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 158: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 159: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 160: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 161: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 162: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 163: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 164: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 165: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 166: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 167: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 168: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 169: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 170: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 171: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 172: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 173: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 174: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 175: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 176: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 177: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 178: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 179: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 180: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 181: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 182: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 183: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 184: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 185: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 186: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 187: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 188: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 189: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 190: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 191: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 192: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 193: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 194: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 195: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 196: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 197: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 198: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 199: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] * 200: [{"id": 1, "nama": "A"}, {"id": 2, "nama": "B"}] </pre>

Skenario atau Langkah	Prediksi	Hasil Aktual	Bukti dari Console, Network, atau Antarmuka
Array kosong []	#status akan menampilkan keterangan “Data Kosong”, karena tidak terdapat data yang dapat ditampilkan dari JSON.	Berhasil memanggil JSON dengan status HTTP 200, namun outputnya adalah #status menampilkan keterangan “Data Kosong”, karena tidak terdapat data yang dapat ditampilkan dari JSON.	
Path salah atau JSON rusak	Akan terdapat console error yang menunjukkan bahwa file yang dipanggil tidak ditemukan.	Terdapat console error not found ketika pemanggilan json, dengan output status 404 not found.	
Klik berulang saat loading	Pengguna tidak dapat melakukan klik berulang saat loading karena button sudah otomatis disabled sesudah diklik.	Pengguna tidak dapat melakukan klik lagi setelah pertama kali klik button, karena button sudah disabled semenjak pertama diklik hingga load selesai.	

State	Apa yang tampil	Kapan terjadi	Bukti
idle	Keterangan pada #status menampilkan “Belum ada proses”.	Ketika web berhasil dimuat, dan belum ada aksi didalamnya.	
loading	Keterangan pada #status menampilkan “Membuat data...”.	Ketika button “Muat data” diklik, dan terjadi proses pengambilan data dari JSON.	
success	Keterangan pada #status menampilkan “3 data tampil”, serta terdapat beberapa data yang diambil dari JSON ditampilkan dibawah button.	Ketika pengambilan data dari JSON berhasil, dan sukses ditampilkan dalam website.	

State	Apa yang tampil	Kapan terjadi	Bukti
empty	Keterangan pada #status menampilkan “Data kosong”.	Ketika sukses melakukan pengambilan data dari JSON, namun tidak terdapat satupun data dalam JSON tersebut.	
error	Keterangan pada #status menampilkan keterangan error.	Terjadi ketika terdapat kesalahan pada code, lokasi json tidak ditemukan, atau ketika terdapat kesalahan pada format penulisan JSON.	

Mengapa delay 0 pada setTimeout tetap tidak membuat callback berjalan sebelum kode sinkron selesai?

Karena Event Loop hanya akan menjalankan tugas dari Task Queue (termasuk setTimeout) setelah semua kode sinkron di Call Stack selesai dieksekusi secara keseluruhan.

Apa perbedaan antara error HTTP atau kesalahan path dengan error parsing JSON berdasarkan bukti yang Anda peroleh?

Error HTTP/Path: Terjadi gagal saat mengambil data (misal: salah path/URL atau server mati), sehingga status respons gagal (seperti 404).

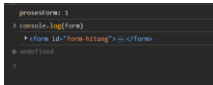
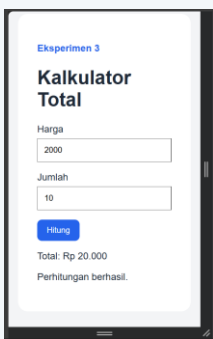
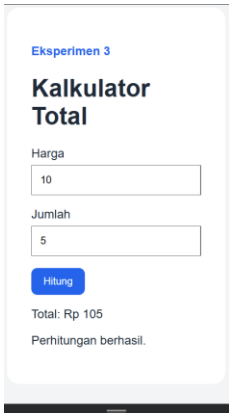
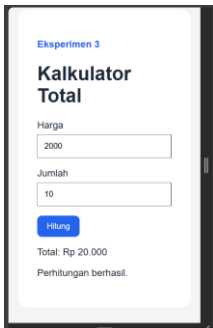
Error Parsing JSON: Pengambilan data sukses, tetapi format isinya salah (bukan JSON yang valid), sehingga memicu `SyntaxError` saat di- parsing menggunakan `.json()`.

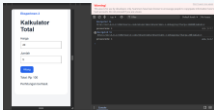
Mengapa finally cocok untuk mengaktifkan kembali tombol?

Karena blok `.finally()` selalu dieksekusi di akhir, baik proses pengambilan data berhasil (`.then`) maupun gagal (`.catch`). Ini menghindari penulisan ulang kode pengaktifan tombol di kedua tempat tersebut.

A4. Eksperimen Diagnosis Error JavaScript

Metode Diagnosis Metode diagnosis dimulai dengan mencatat gejala dan hasil yang diharapkan, mereproduksi masalah, membaca pesan error pertama, memeriksa nilai atau selector, menyusun dugaan penyebab, melakukan satu perubahan, kemudian menguji ulang hasil perbaikan.

Kasus	Gejala dan Hasil yang Diharapkan	Pesan Error atau Nilai Aktual	Dugaan Penyebab	Perbaikan	Bukti Uji Ulang
A · Selector	Form tidak merespons tombol submit.	app.js:29 Uncaught TypeError: Cannot read properties of null (reading 'addEventListener')	Selector document.querySelector('#form-hitung') pada JavaScript menghasilkan null karena atribut ID di file index.html diubah menjadi form-total.	Mengembalikan ID pada tag form di HTML dari id="form-total" menjadi id="form-hitung" agar cocok dengan selector di JS.	 console.log(form) menghasilkan elemen <form id="form-hitung"> utuh dan prosesForm: 1 muncul di console.
B · Nama variabel	Proses eksekusi terhenti dan hasil total tidak dapat ditampilkan saat submit form.	app.js:19 Uncaught ReferenceError: hargaInput is not defined	Terdapat typo kapitalisasi (JavaScript bersifat case-sensitive). Variabel dideklarasikan sebagai hargaInput (I besar) tetapi dipanggil sebagai hargaInput (i kecil).	Memperbaiki / mengoreksi pemanggilan variabel menjadi hargaInput.value di dalam blok prosesForm.	 Fungsi berjalan lancar (muncul pesan berhasil) dan tidak ada teks error merah di console.
C · Tipe data dan operator	Hasil total adalah penggabungan teks bukan operasi matematika. (Misal: harga 10 & jumlah 5 menjadi 105).	 Nilai aktual yang keluar adalah string '105'. typeof harga dan typeof jumlah terdeteksi sebagai 'string'.	Nilai bawaan atribut .value form selalu berupa string. Jika tipe string dikenakan operator +, ia akan menggabungkan teks, bukan menjumlahkan angka.	Menggunakan kembali fungsi Number() pada nilai inputan, dan mengembalikan operator + menjadi * di fungsi hitung.	 typeof harga kembali mencetak 'number' dan perhitungan kembali normal (misal 10*5 menjadi 50).

Kasus	Gejala dan Hasil yang Diharapkan	Pesan Error atau Nilai Aktual	Dugaan Penyebab	Perbaikan	Bukti Uji Ulang
D · Form reload	Halaman website memuat ulang (reload), input ter-reset, dan log console langsung hilang setiap kali menekan Hitung.	 Halaman melakukan navigasi URL yang ditandai dengan munculnya query parameter.	Fungsi <code>event.preventDefault()</code> yang bertugas mencegah perilaku bawaan form HTML (reload browser saat dikirim) telah dikomentari / dinonaktifkan.	Menghilangkan <code>comment</code> dan memanggil kembali fungsi <code>event.preventDefault()</code> pada baris awal <code>prosesForm</code> .	 Halaman form statis tidak ter-reload, URL browser tidak berubah, dan hitungan log tetap menetap di console.
E · Handler ganda	Fungsi pemrosesan <code>prosesForm</code> dijalankan sebanyak dua kali meskipun tombol submit hanya diklik sekali.	 Log hitungan bertambah dua lompatan (misal: langsung ke angka 2, 4, dst) dalam sekali eksekusi form.	Terdapat dua event listener submit yang sama-sama terpasang dan memanggil fungsi <code>prosesForm</code> secara beruntun.	Menghapus / menghilangkan blok kode <code>form.addEventListener</code> tambahan di akhir baris program.	 Nilai <code>console.count('prosesForm')</code> hanya murni bertambah satu angka per kali submit ditekan.

Kasus paling membantu pemahaman saya dan mengapa:

Kasus C (Tipe data / Operator): Kasus ini paling menyadarkan saya bahwa semua nilai input HTML otomatis berupa String. Jika lupa dikonversi ke Number, operator + malah akan menempelkan teks (misal 10+5 = 105), bukan menjumlahkannya secara matematika.

B. Pertanyaan Pemahaman

B1. Logika, Function, Array, dan Object

Kapan let lebih tepat digunakan daripada const?

Saat kita tahu nilai variabel tersebut akan berubah/ditiban di kemudian waktu (misalnya di dalam perulangan / looping).

Mengapa nilai dari elemen input perlu dikonversi sebelum perhitungan?

Karena nilai bawaan input HTML selalu berupa String (teks). Tanpa konversi, perhitungan matematika tidak bisa berjalan benar.

Mengapa kondisi kategori atau diskon dengan batas terbesar diperiksa lebih dahulu?

Agar kondisi yang paling sulit dicapai tidak terlewat / tertiban oleh kondisi di bawahnya yang lebih mudah.

Apa perbedaan parameter, argument, dan return?

Parameter adalah wadah di definisi fungsi. Argument adalah nilai asli yang dioper saat pemanggilan fungsi. Return adalah hasil akhir yang dikembalikan fungsi.

Mengapa function logika sebaiknya dipisahkan dari function yang mengubah tampilan?

Agar kode lebih rapi, fungsi logika bisa dipakai ulang, dan kita bisa mengecek logika tanpa harus bergantung pada elemen HTML.

Mengapa variabel lokal tidak dapat diakses dari luar function?

Karena JavaScript mengisolasi ruang lingkup (scope) variabel di dalam fungsi agar tidak bentrok dengan variabel di luar.

Mengapa kumpulan entitas dengan struktur sama cocok disimpan sebagai array of objects?

Array memudahkan kita melakukan perulangan (loop), sedangkan Object menyimpan detail informasinya secara terstruktur dalam satu kesatuan.

Apa akibat mengakses indeks array yang tidak tersedia?

Akan mengembalikan nilai undefined.

B2. DOM, Event, dan Validasi Form

Apa yang dikembalikan querySelector ketika selector tidak menemukan elemen?

Mengembalikan nilai null.

Mengapa classList lebih tepat untuk perubahan visual daripada menulis style satu per satu di JavaScript?

Lebih rapi, kodenya pendek, dan memisahkan tugas desain (CSS) dengan logika (JS).

Apa hubungan antara addEventListener, event handler, dan perubahan DOM?

`addEventListener` memantau aksi user, memicu event handler (fungsi) untuk berjalan, lalu fungsi tersebut melakukan perubahan DOM (update layar).

Mengapa atribut `defer` digunakan ketika `app.js` dipanggil dari `head`?

Agar HTML diselesaikan muatannya lebih dulu oleh browser, baru eksekusi Javascript dijalankan (mencegah error elemen tidak ditemukan).

Mengapa `textContent` digunakan untuk teks biasa atau data input pengguna?

Untuk alasan keamanan. Menghindari script berbahaya dieksekusi

Apa fungsi `event.preventDefault()` pada submit form?

Untuk menahan sifat bawaan elemen form yang otomatis me-reload halaman browser ketika di-submit.

Mengapa `trim` digunakan sebelum memvalidasi input teks?

Untuk menghapus spasi kosong yang tidak sengaja terketik di awal dan akhir, agar spasi kosong tidak lolos sebagai input valid.

Mengapa validasi sisi klien tetap tidak menggantikan validasi server?

Karena validasi dari browser bisa dengan mudah dimatikan atau diakali oleh user, keamanan utama harus selalu ada di server.

B3. Asynchronous JavaScript

Apa perbedaan state Promise `pending`, `fulfilled`, dan `rejected`?

`Pending`: Proses sedang berjalan. `Fulfilled`: Proses sukses. `Rejected`: Proses gagal/error.

Bagian mana yang ditunda oleh `await` dan bagian mana yang tetap dapat merespons pengguna?

`await` menunda eksekusi kode di dalam fungsi `async` itu saja, sedangkan interaksi/klik pengguna di layar HTML tetap bisa direspons.

Mengapa `fetch` dengan status HTTP 404 tidak otomatis masuk ke `catch`?

Karena `fetch` menganggap 404 (Not Found) sebagai "berhasil nyambung ke server", ia hanya lari ke `catch` jika internet putus sepenuhnya.

Apa fungsi `response.ok` sebelum `response.json()` dipanggil?

Untuk memastikan status koneksi benar-benar sukses (kode 200-an) sebelum mencoba membongkar isi datanya.

Apa perbedaan state `empty` dan `error` dari sudut pandang pengguna?

`Empty`: Data berhasil dicari tapi hasilnya kosong. `Error`: Ada masalah koneksi atau server rusak.

Mengapa tombol dinonaktifkan selama proses permintaan data berlangsung?

Mencegah user klik tombolnya berkali-kali yang bisa membuat koneksi ke server ganda (membebani server).

Mengapa state `loading` harus selalu diakhiri setelah proses selesai, baik berhasil maupun gagal?

Agar pengguna tidak terjebak terus-terusan melihat animasi `loading` (biasanya diamankan di blok `.finally()`).

Bagaimana Console dan Network saling melengkapi ketika mendiagnosis `fetch`?

Tab `Network` melihat detail aliran data/internetnya, sedangkan tab `Console` melihat bagaimana kode JS menangani data tersebut.

B4. Debugging

Mengapa error pertama pada Console diperiksa sebelum error berikutnya?

Karena error kedua dan seterusnya sering kali hanya efek domino akibat error yang paling pertama.

Apa perbedaan gejala, penyebab, dugaan, dan bukti?

Gejala: Apa yang error/aneh di layar. Penyebab: Kesalahan aslinya di kode. Dugaan: Hipotesa awal kita. Bukti: Hasil pembuktian setelah diuji ulang.

Bagaimana typeof membantu diagnosis kesalahan perhitungan?

Untuk memastikan apakah kita sedang menghitung antar-angka ('number') atau tanpa sengaja menggabungkan teks ('string').

Bagaimana console.count membuktikan handler berjalan lebih dari sekali?

Karena ia otomatis menambah angka (1, 2, 3...) tiap kali fungsinya terlewati dalam satu kali klik.

Mengapa hanya satu perubahan dilakukan pada setiap percobaan perbaikan?

Agar kita tahu secara pasti perbaikan/baris mana yang benar-benar memecahkan masalahnya.

Apa bukti minimum bahwa perbaikan benar-benar berhasil dan bukan hanya menyembunyikan gejala?

Hasilnya kembali sukses seperti yang diharapkan dan tidak ada lagi teks error merah di console.

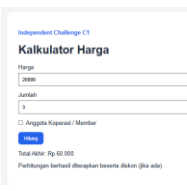
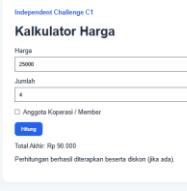
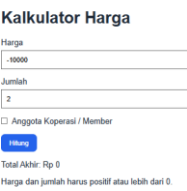
C. Independent Challenge

C1. Kalkulator Harga

Gunakan program pada Console dan function yang memiliki nilai kembalian. Lakukan sedikitnya lima kasus pengujian, termasuk nilai batas Rp100.000, Rp200.000, serta satu input yang tidak valid.

Keputusan	Isian Mahasiswa
Data input dan tipe yang digunakan	• harga (tipe: Number) • jumlah (tipe: Number) • isAnggota (tipe: Boolean)
Function yang dibuat dan tanggung jawabnya	• hitungTotalHarga(harga, jumlah, isAnggota): Menerima input, memvalidasi data, menghitung harga awal, memanggil fungsi diskon, dan mengembalikan total akhir. • hitungDiskon(totalAwal, isAnggota): Menghitung dan menentukan total persentase diskon yang didapat.
Aturan diskon 10%, 20%, tambahan anggota 5%	• Jika total awal $\geq$ Rp200.000, diskon = 20% • Jika total awal $\geq$ Rp100.000 (tapi $<$ Rp200.000), diskon = 10% • Jika total awal $<$ Rp100.000, diskon = 0% • Jika isAnggota bernilai true, tambahkan diskon ekstra +5% ke diskon yang sudah didapat.
Cara membatasi total diskon maksimal 25%	Menjumlahkan total persentase diskon, lalu membatasinya dengan menggunakan <code>Math.min(totalDiskon, 25)</code> agar persentase diskon tidak pernah melebihi 25% walau kondisinya terpenuhi semua.

Keputusan	Isian Mahasiswa
Cara menangani harga atau jumlah yang kurang dari atau sama dengan 0	Menggunakan seleksi kondisi awal (if (harga <= 0))

Kasus Uji	Harga	Jumlah	Status Anggota	Ekspektasi	Aktual	Status
1. Normal (Tanpa diskon)	20.000	3	False	60.000		☒ Lulus ☐ Gagal
2 · batas Rp100.000	25.000	4	False	90.000 (Diskon 10%)		☒ Lulus ☐ Gagal
3 · batas Rp200.000	50.000	4	True	150.000 (Diskon 20% + 5% = 25%)		☒ Lulus ☐ Gagal
4. Maksimal Diskon	100.000	5	True	375.000 (Diskon maksimal tetap 25%)		☒ Lulus ☐ Gagal
5 · tidak valid	-10.000	2	False	Input tidak valid / Error / 0		☒ Lulus ☐ Gagal

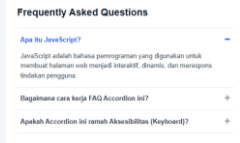
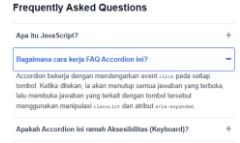
Jelaskan kasus nilai batas yang paling penting beserta alasannya:

Kasus nilai batas yang paling penting adalah saat total harga berada tepat di angka Rp100.000 dan Rp200.000, serta ketika input bernilai \$0\$ atau negatif. Alasannya:

- Angka Rp100.000 dan Rp200.000 merupakan titik batas di mana logika program mengalami perubahan (transisi diskon dari 0% ke 10%, dan 10% ke 20%). Pengujian di batas ini sangat penting untuk mencegah Off-By-One Error (misalnya keliru menggunakan operator $>$ alih-alih $\geq$), yang bisa membuat pembeli kehilangan hak diskonnya secara tidak adil.
- Nilai batas 0 (dan negatif) penting untuk diuji guna memastikan program dapat mendeteksi serta menangkal input yang tidak logis di awal, sehingga aplikasi tidak menghasilkan angka perhitungan yang merugikan (minus).

C2. FAQ Accordion

- ☑ Tiga item FAQ dibuat dengan struktur semantik.
- ☑ Pertanyaan menggunakan button, bukan div yang diberi click handler.
- ☑ Event dipasang dengan addEventListener dan tampilan berubah melalui classList.
- ☑ aria-expanded berubah sesuai keadaan.
- ☑ Hanya satu jawaban terbuka pada satu waktu.
- ☑ Dapat digunakan melalui keyboard tanpa library accordion.

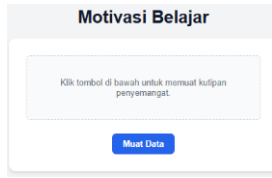
Skenario Uji	Ekspektasi	Hasil Aktual	Bukti
Halaman awal	Semua jawaban tertutup	Sesuai ekspektasi / Lulus (semua konten memiliki class collapse)	
Klik FAQ pertama	Jawaban 1 terbuka; aria-expanded=true	Sesuai ekspektasi / Lulus	
Klik FAQ kedua	Jawaban 2 terbuka; FAQ 1 tertutup	Sesuai ekspektasi / Lulus	
Klik FAQ aktif lagi	State berubah secara konsisten	Sesuai ekspektasi / Lulus (kembali tertutup)	
Keyboard	Navigasi menggunakan Tab serta aktivasi menggunakan Enter atau Space berfungsi	Sesuai ekspektasi / Lulus	

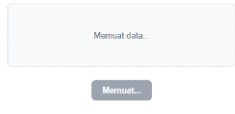
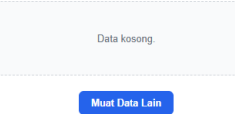
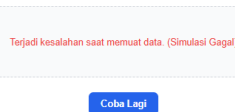
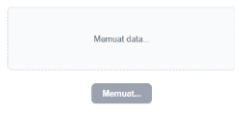
Bagaimana Anda memastikan hanya satu jawaban terbuka pada satu waktu?

Dengan menggunakan teknik perulangan (loop) pada event listener saat ada klik. Ketika sebuah tombol accordion diklik, sistem tidak langsung membukanya. Pertama, ia akan melakukan looping untuk mengembalikan atribut `aria-expanded` menjadi `false` dan menimpa class penampil (`show`) menjadi penyembunyi (`collapse`) pada seluruh elemen accordion. Hal ini menjamin statusnya "reset" (tertutup semua). Setelah semuanya dipastikan tertutup, barulah sistem mengecek status awal tombol yang barusan ditekan; jika sebelumnya sedang tertutup, maka tombol tersebut akan dibuka (`aria-expanded="true"` dan menambahkan class `show`).

C3. Async Data Loader

Aspek Desain	Keputusan Mahasiswa
Sumber data JSON lokal	Saya membuat file <code>quotes.json</code> di dalam folder <code>data</code> yang berisi array of string (kutipan/tips belajar). Data ini akan dimuat menggunakan fungsi <code>fetch()</code> .
Delay buatan selama 500 sampai 1500 ms	Delay ini dikelola menggunakan fungsi <code>setTimeout()</code> yang dimasukkan ke dalam objek Promise buatan. Nilai milidetiknya menggunakan fungsi <code>Math.random()</code> yang diatur antara angka 500 hingga 1500.
Cara memilih satu item secara acak	Setelah JSON di-parse ke bentuk Array JavaScript, program akan memanggil metode <code>Math.floor(Math.random() * array.length)</code> untuk mendapatkan nomor indeks acak dari 0 hingga elemen terakhir.
Simulasi kegagalan sebesar 30%	Sebelum memanggil HTTP Request (<code>fetch</code>), program mengecek <code>Math.random() < 0.3</code> . Jika bernilai <code>true</code> (30% peluang terjadi), Promise akan dikembalikan sebagai <code>Reject</code> (Gagal) dan dilempar ke blok <code>catch</code> .
State loading, empty, error, dan mekanisme percobaan ulang	Menggunakan manipulasi pada DOM via properti <code>classList.toggle('hidden')</code> untuk merender/menghilangkan elemen penampung state (misal: <code>ID loading-state</code> , <code>empty-state</code>). Tombol action akan berubah teks menjadi "Coba Lagi" saat blok <code>catch</code> dieksekusi.
Cara mencegah klik berulang agar hasil render tetap konsisten	Di awal eksekusi handler, saya menset properti <code>button.disabled = true</code> agar tombol tidak bisa diklik. Status <code>disabled</code> baru diubah ke <code>false</code> hanya pada blok <code>finally</code> (setelah operasi selesai terlepas dari berhasil atau gagal).

Skenario Uji	Ekspektasi	Hasil Aktual	Bukti
Pemuatan normal	Teks kutipan tampil secara acak, tombol kembali aktif.	Lulus / Sesuai Ekspektasi	

Skenario Uji	Ekspektasi	Hasil Aktual	Bukti
Loading terlihat	Selama ± 1 detik, tombol tidak dapat diklik (disabled), muncul teks "Memuat data..."	Lulus / Sesuai Ekspektasi	Motivasi Belajar 
Data kosong	(Jika simulasi file JSON dikosongkan) Tampil pesan "Data kosong" di layar.	Lulus / Sesuai Ekspektasi	Motivasi Belajar 
Simulasi gagal	Sekitar 3 dari 10 klik akan memicu teks peringatan error warna merah.	Lulus / Sesuai Ekspektasi	Motivasi Belajar 
Tombol Coba Lagi	Saat simulasi gagal, tombol tertulis "Coba Lagi". Jika ditekan, ia berusaha fetch ulang.	Lulus / Sesuai Ekspektasi	Motivasi Belajar 
Klik berulang	Saat menekan dengan intens dan cepat, UI tetap aman karena tombol terkunci sesaat (disabled).	Lulus / Sesuai Ekspektasi	Motivasi Belajar 

Jelaskan alur Promise dan proses asinkron sejak tombol diklik hingga data berhasil ditampilkan atau error ditangani.

1. Ketika tombol "Muat Data" diklik, ia akan menjalankan Event Listener berbasis fungsi async.
2. Properti elemen langsung diperbarui (state loading menyala dan tombol di-disable).
3. Fungsi fetchQuotes() dipanggil dan ditunggu (await). Fungsi ini mereturn sebuah custom Promise.
4. Di dalam Promise ini, sebuah waktu jeda buatan dibuat menggunakan setTimeout (500–1500ms).
5. Begitu waktu jeda selesai, sistem melempar "dadu" probabilitas. Jika tebakkan 30% terjadi, ia akan mengeksekusi metode reject(new Error("...")) tanpa perlu melakukan fetch.
6. Jika tidak masuk 30%, ia akan menjalankan fetch('data/quotes.json'). Hasil dari fetch di-parsing ke .json(), kemudian men-trigger pemanggilan resolve(data).
7. Jika resolve dipanggil, Promise kembali ke blok pemanggil secara utuh, menampilkan datanya ke textContent (state success).
8. Jika reject dipanggil di tengah jalan (entah karena simulasi gagal atau JSON tak ditemukan), status akan tertangkap oleh blok catch(error) yang akan menampilkan pesan error (state error).
9. Paling akhir, blok finally dieksekusi untuk mereset fungsi tombol menjadi aktif (disabled = false) dan bisa diklik kembali.

D. Tugas Praktikum

D1. Task 1 — JavaScript Logic Builder

Target: menggunakan function murni tanpa DOM untuk melakukan validasi nilai, menentukan kategori dan status, serta membentuk ringkasan data.

Function	Tanggung Jawab	Input	Nilai Kembali atau Keluaran
validasiNilai	Menentukan apakah nilai yang diberikan valid. Aturannya: tipe data berupa number, bersifat finite (bukan NaN/Infinity), serta di antara rentang angka 0 hingga 100.	nilai (Number/Campuran)	Boolean (true jika valid, false jika tidak valid)
tentukanKategori	rentang angka berjenjang lalu memetakan input nilai angka (0-100) ke indeks huruf (A, B, C, D). Menangani kasus nilai tidak valid dengan me-return nilai null.	nilai (Number)	String berupa indeks ("A", "B", "C", atau "D"), atau Null.
tentukanStatus	Mengkategorikan batas lulus atau gagal berdasarkan nilai angka. Jika tidak valid, mengembalikan pesan pemberitahuan "Data tidak valid".	nilai (Number)	String ("Lulus", "Tidak lulus", atau "Data tidak valid").
buatRingkasan	Menghimpun (membungkus) nilai awal beserta hasil kategori dan kelulusannya ke dalam satu buah Object secara terstruktur tanpa menulis ulang logika perhitungan.	nama (String), nilai (Number)	Object ({ nama, nilai, kategori, status })

Kasus	Input Nilai	Kategori Diharapkan	Status Diharapkan	Hasil Aktual	Status
1	0	D	Tidak lulus	Sesuai (Kategori: D, Status: Tidak lulus)	☑
2	59	D	Tidak lulus	Sesuai (Kategori: D, Status: Tidak lulus)	☑
3	60	C	Lulus	Sesuai (Kategori: C, Status: Lulus)	☑
4	69	C	Lulus	Sesuai (Kategori: C, Status: Lulus)	☑
5	70	B	Lulus	Sesuai (Kategori: B, Status: Lulus)	☑
6	84	B	Lulus	Sesuai (Kategori: B, Status: Lulus)	☑

Kasus	Input Nilai	Kategori Diharapkan	Status Diharapkan	Hasil Aktual	Status
7	85	A	Lulus	Sesuai (Kategori: A, Status: Lulus)	☑
8	101	null	Data tidak valid	Sesuai (Kategori: null, Status: Data tidak valid)	☑
9	"80"	null	Data tidak valid	Sesuai (Kategori: null, Status: Data tidak valid)	☑

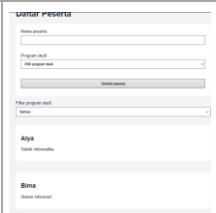
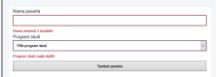
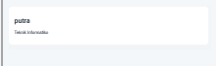
Mengapa 59, 60, 69, 70, 84, dan 85 merupakan boundary value penting?

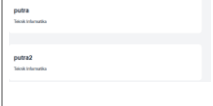
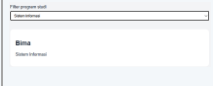
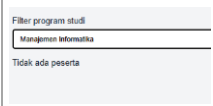
Nilai-nilai ini dikenal sebagai Boundary Values (Nilai Batas). Menguji di titik batas sangat krusial untuk memastikan tidak terjadi cacat Off-By-One Error. Seringkali, programmer salah memakai operator perbandingan, misal memakai > yang seharusnya >= (menyebabkan angka 60 jadi dianggap tidak lulus padahal tepat berada di batasnya).

Tuliskan perubahan nilai batas yang diuji, prediksi dampaknya, dan hasil pengujian yang diperoleh.

1. Perubahan yang diuji: Mengubah batas nilai kategori C pada fungsi tentukanKategori dari if (nilai >= 60) menjadi if (nilai > 60).
2. Prediksi dampak: Kasus uji dengan input nilai persis 60 (seperti pada kasus "Citra") tidak akan lagi memenuhi syarat untuk masuk ke kategori C, melainkan akan merosot ke blok pengembalian default di bawahnya, yaitu kategori 'D'.
3. Hasil pengujian yang diperoleh: Setelah dijalankan ulang, prediksi terbukti benar. Pada tabel console, input nilai 60 menghasilkan kategori 'D' (bukan 'C' seperti seharusnya). Hal ini membuktikan bahwa kesalahan kecil pada operator perbandingan (memakai > alih-alih >=) di titik batas (boundary value) dapat mengubah logic program secara fatal dan merugikan sistem penilaian. Aturan kemudian dipulihkan kembali ke awal (>=).

D2. Task 2 — DOM dan Form Controller

Skenario	Tindakan	Hasil yang Diharapkan	Hasil Aktual	Bukti
Awal	Muat halaman	Dua kartu peserta tampil	Sesuai / Lulus	
Kosong	Submit tanpa isi	Dua pesan error; data tidak bertambah	Sesuai / Lulus	
Nama pendek	Nama 2 karakter	Error hanya pada nama	Sesuai / Lulus	
Valid	Isi semua field	Satu kartu baru ditampilkan	Sesuai / Lulus	

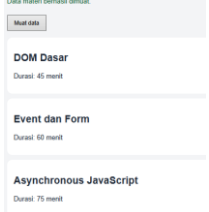
Skenario	Tindakan	Hasil yang Diharapkan	Hasil Aktual	Bukti
Submit kedua	Tambah peserta lain	Data lama tidak terduplikasi	Sesuai / Lulus	
Filter ada	Pilih satu prodi	Hanya program studi tersebut yang ditampilkan	Sesuai / Lulus	
Filter kosong	Pilih Manajemen Informatika	Pesan "Tidak ada peserta" ditampilkan	Sesuai / Lulus	
Keyboard	Tab lalu Enter	Seluruh kontrol dapat digunakan melalui keyboard	Sesuai / Lulus	

Keputusan Teknis	Penjelasan / Bukti
Cara menggunakan createElement dan textContent	Digunakan pada fungsi buatKartuPeserta(): memanggil document.createElement('article') kemudian mengisi datanya menggunakan atribut textContent (h2.textContent = item.nama;) alih-alih innerHTML demi keamanan dari injeksi kode
Cara replaceChildren mencegah duplikasi	Digunakan pada awal logika renderPeserta() di dalam DOM. Fungsi daftar.replaceChildren(fragment) secara otomatis akan mengosongkan semua isi/node lama di dalam elemen daftar sebelum menaruh kumpulan node yang baru, sehingga data lama tidak menumpuk dengan yang baru.
Cara validasi dipisahkan dari render DOM	Logika pemrosesan nilai dikapsulasi di fungsi murni validasiPeserta(calon) yang hanya bertugas mendeteksi benar/salah lalu mereturn Object state. Fungsi ini tidak ikut campur mengubah warna tombol atau memunculkan peringatan, karena tugas tersebut (manipulasi DOM) diserahkan murni kepada penangkap event (addEventListener). Praktik ini disebut Separation of Concerns.
Cara aria-invalid diperbarui	Properti ini dikontrol di dalam blok form.addEventListener('submit'). Ia akan diberikan nilai 'true' dengan setAttribute('aria-invalid', 'true') jika fungsi validasiPeserta menyatakan error pada input tersebut. Sebaliknya, atribut diset ke 'false' (atau dihapus) jika validasinya lolos.
Cara filter menjaga array peserta asli	Pada bagian pengecekan select dropdown filter, saya menggunakan metode peserta.filter(...). Method array ini bersifat immutable, artinya ia menciptakan susunan Array baru tanpa merusak memori dari variabel peserta aslinya. Dengan ini, bila pengguna mematikan filter (pilih "Semua"), kita cukup me-render ulang Array asli.

Kesalahan program yang paling penting yang ditemukan pada Task 2 serta cara memperbaikinya:

1. Halaman me-refresh sendiri dan data hilang setiap kali tombol "Tambah" di-klik. Cara memperbaiki: Menambahkan event.preventDefault() di awal fungsi handler submit agar browser tidak melakukan reload halaman.
2. Kesalahan 2: Kartu data baru menumpuk dengan kartu lama (terduplikasi ganda) setiap kali fungsi render dipanggil. Cara memperbaiki: Menggunakan metode daftar.replaceChildren() di awal fungsi renderPeserta() untuk mengosongkan antarmuka (DOM) sebelum merender ulang datanya.

D3. Task 3 — Asynchronous JavaScript Data Loader

Skenario	Cara Membuat	State dan Hasil yang Diharapkan	Hasil Aktual	Bukti
Idle	Muat halaman	Belum ada permintaan; tombol aktif	Sesuai / Lulus	Materi JavaScript Tekan tombol untuk memuat data. 
Loading	Klik Muat data	Pesan loading; tombol dinonaktifkan	Sesuai / Lulus	Materi JavaScript Memuat data ... 
Success	JSON valid	Tiga kartu tampil; status 200	Sesuai / Lulus	Materi JavaScript Data materi berhasil dimuat. 
Empty	JSON = []	Pesan data kosong; bukan error	Sesuai / Lulus	Materi JavaScript Tidak ada data materi (kosong). 
Path salah	Ubah URL fetch	State error dan status 404 terlihat pada Network	Sesuai / Lulus	Materi JavaScript Gagal memuat data. Silakan periksa koneksi atau console untuk detailnya. 
JSON rusak	Rusak sintaks JSON	Parsing gagal dan state berubah menjadi error	Sesuai / Lulus	Materi JavaScript Gagal memuat data. Silakan periksa koneksi atau console untuk detailnya. 
Server mati	Hentikan local server	Fetch ditolak; antarmuka menampilkan error	Sesuai / Lulus	Console mencatat TypeError: Failed to fetch. Status koneksi terputus.
Klik ganda	Klik tombol dua kali dengan cepat	Satu proses; daftar tidak ganda	Sesuai / Lulus	Tab Network hanya merekam 1 buah request materi.json, data tidak duplikat di UI.

Pemeriksaan melalui Network dan Console	Temuan
URL permintaan dan status HTTP	URL mengarah ke /data/materi. Saat file ada, statusnya adalah 200 OK. Saat path disalahkan, statusnya menjadi 404 Not Found.
Respons JSON	Pada tab Response/Preview di dalam panel Network, terlihat wujud asli kembalian data berupa Array of Objects (berisi id, judul, durasi).
Pesan error akibat path atau status HTTP	Console mencetak tangkapan Error buatan dari hasil pengecekan kondisional if (!response.ok) dengan pesan seperti HTTP 404.
Pesan error parsing	Console menangkap dan mencetak SyntaxError (seperti Unexpected token) karena response.json() gagal mem-parsing/mengurai format teks yang rusak menjadi object JavaScript.

Pemeriksaan melalui Network dan Console	Temuan
Jumlah permintaan ketika tombol diklik dua kali dengan cepat	Panel Network membuktikan bahwa hanya ada 1 permintaan (request) fetch yang berjalan, karena property disabled = true pada tombol langsung bekerja dan mencegah pemicuan klik fungsi kedua kalinya.

Jelaskan urutan state yang Anda amati sejak tombol diklik hingga seluruh proses selesai:

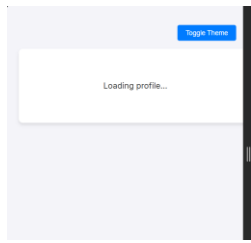
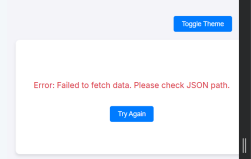
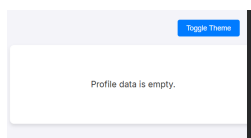
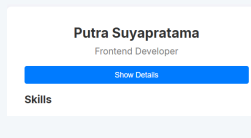
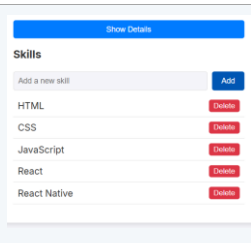
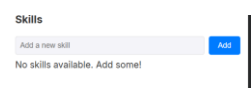
1. Kondisi halaman awal adalah Idle.
2. Saat tombol diklik, state masuk ke fase Loading: Tombol dinonaktifkan (kunci ganda) dan UI memunculkan pesan memuat. Promise fetch mulai diinisiasi (fase pending).
3. Alur menunggu respons secara asynchronous (await). Saat response datang, status HTTP divalidasi.
4. Apabila berhasil, data dirender ke dalam Grid (masuk state Success), atau jika array berjumlah 0 maka masuk state Empty.
5. Apabila gagal (koneksi mati, URL salah, JSON cacat), eksekusi berpindah ke dalam blok catch dan masuk ke state Error (memunculkan pesan peringatan di UI & menampilkan tombol Coba Lagi).
6. Terakhir, blok finally berjalan secara mutlak untuk membersihkan status loading dan mengaktifkan kembali (lepas kunci) tombol aksi.

Mengapa Coba Lagi sebaiknya memanggil function pemuatan yang sama?

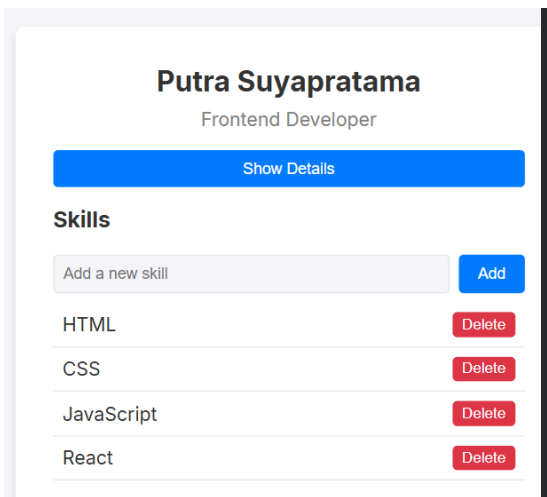
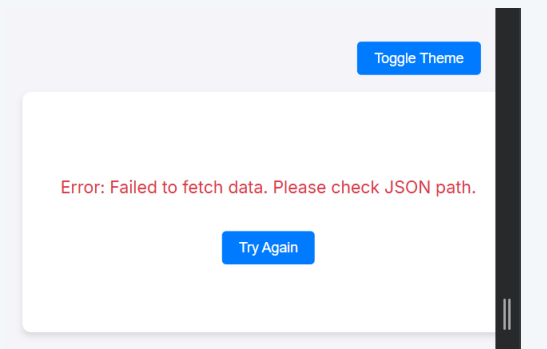
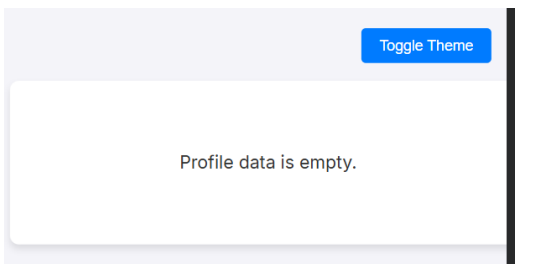
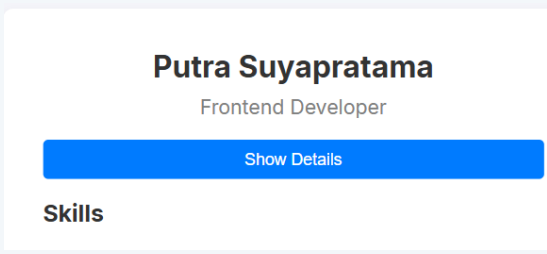
Untuk mencegah duplikasi kode. Karena fungsi muatData() sudah dirancang lengkap dengan reset state, urutan dari proses loading awal hingga finally akhir, maka menggunakan ulang fungsi ini untuk "Coba Lagi" memastikan perilaku komponen sangat konsisten tanpa harus menduplikasi logika penarikan datanya.

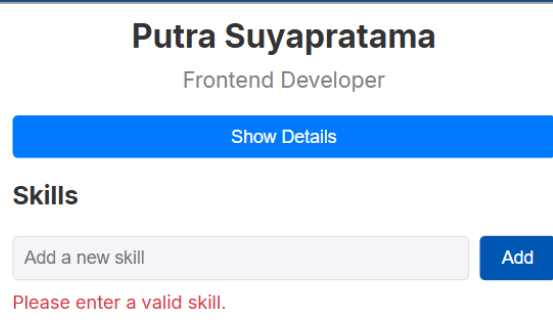
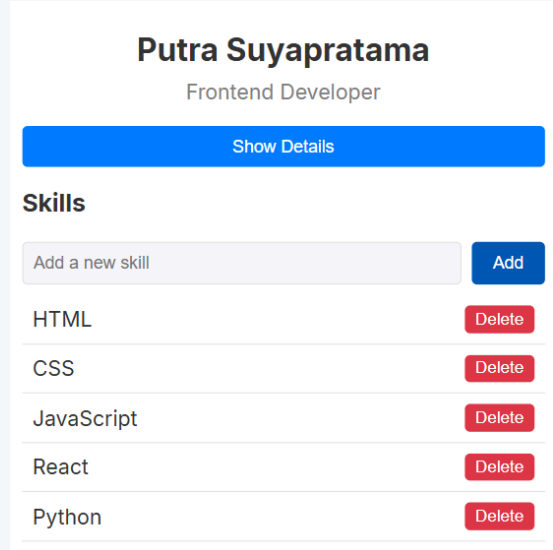
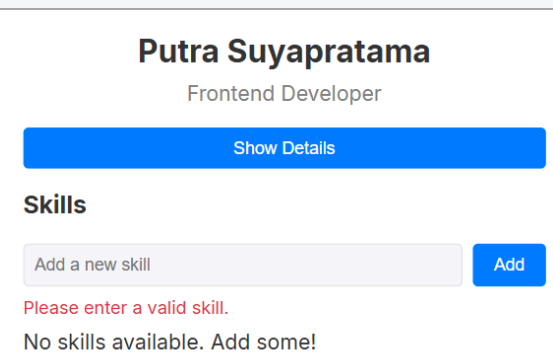
D4. Homework — Interactive Profile Card

Fitur atau Kondisi	Status	Bukti dan Catatan
Profil dan keterampilan awal dimuat dari data/profile.json	☑ Selesai	Data berhasil di-fetch dan ditampilkan di halaman web.
State loading, empty, error, dan Coba Lagi tersedia	☑ Selesai	State loading muncul saat memuat data, error muncul saat path salah, empty muncul jika data kosong.
Detail toggle dengan classList dan aria-expanded	☑ Selesai	Menggunakan classList.toggle('hidden') dan mengubah atribut aria-expanded.
Tema dapat diganti	☑ Selesai	Toggle tema dari terang ke gelap (dark-theme) berhasil diimplementasikan.
Form penambahan keterampilan memvalidasi input kosong	☑ Selesai	Muncul pesan error jika input kosong dan data tidak ditambahkan.
Setiap keterampilan dapat dihapus	☑ Selesai	Setiap item skill memiliki tombol "Delete".
Klik berulang tidak menggandakan data	☑ Selesai	Validasi mencegah input skill yang sudah ada.
README menjelaskan cara menjalankan local server	☑ Selesai	Tersedia opsi Live Server, Node.js, dan Python.

Skenario	Ekspektasi	Hasil Aktual	Bukti
Halaman dibuka	Loading lalu profil tampil tanpa error	Halaman menampilkan "Loading profile...", lalu menampilkan nama "Jane Doe" beserta skills.	
Path JSON salah	Pesan error dan Coba Lagi tampil	Halaman menampilkan "Error: Failed to fetch data..." dengan tombol "Try Again" saat path pada fetch disalahkan sementara.	
JSON kosong	State empty ditampilkan	Menampilkan teks "Profile data is empty." jika profile.json dikosongkan {}.	
Detail diklik dua kali	Terbuka lalu tertutup	Bagian detail muncul (tombol jadi "Hide Details"), lalu tertutup kembali saat diklik lagi.	
Input kosong	Data tidak ditambah; pesan muncul	Muncul pesan merah "Please enter a valid skill." di bawah form.	
Keterampilan yang valid	Item baru muncul satu kali	Menambahkan "React Native", item muncul di daftar skill. Jika diklik dua kali, tidak terduplikasi.	
Semua keterampilan dihapus	Pesan untuk kondisi kosong ditampilkan	Setelah semua dihapus, muncul tulisan "No skills available. Add some!".	

Daftar minimal tujuh tangkapan layar pengujian yang disertakan:

No.	Tangkapan layar	Skenario yang Dibuktikan
1		Refresh halaman, tangkap layar saat halaman menampilkan profil Jane Doe lengkap dengan skill.
2		Ubah sementara fetch('data/profile.json') di app.js menjadi fetch('data/salah.json'), refresh halaman, tangkap layar pesan error. (Kembalikan kode setelah selesai).
3		Ubah sementara isi profile.json menjadi {} (hanya kurung kurawal kosong), refresh halaman, tangkap layar pesan empty. (Kembalikan data seperti semula setelah selesai).
4		Klik tombol "Show Details", tangkap layar ketika bio muncul.

No.	Tangkapan layar	Skenario yang Dibuktikan
5		Kosongkan input form skill, tekan "Add", tangkap layar saat pesan peringatan merah muncul.
6		Tambahkan skill baru (contoh: Python). Tangkap layar. Coba tambah lagi "Python" untuk membuktikan tidak ada data ganda.
7		Klik semua tombol Delete pada daftar skill, tangkap layar ketika pesan "No skills available. Add some!" muncul.

E. Mini Project — Landing Page Interaktif

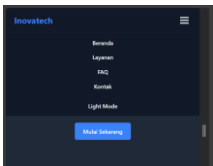
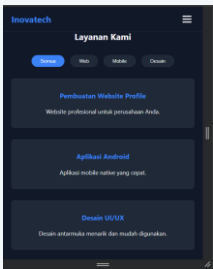
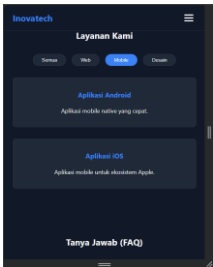
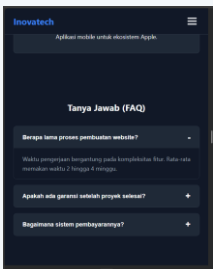
Batas Teknis Mini project menggunakan HTML, CSS, dan vanilla JavaScript. Belum menggunakan fetch, Promise, atau async/await; asynchronous dinilai pada Task 3 dan homework.

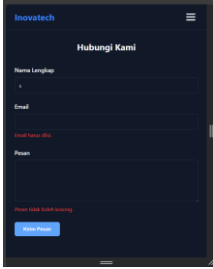
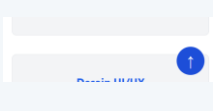
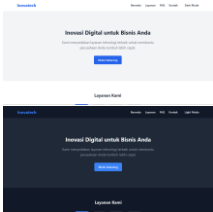
E1. Ringkasan Produk

Aspek	Isian Mahasiswa
Nama atau topik landing page	Inovatech (Solusi Digital Masa Depan)

Aspek	Isian Mahasiswa
Target pengguna	Perusahaan/Bisnis yang membutuhkan layanan digital.
Masalah atau kebutuhan pengguna	Membutuhkan informasi yang jelas tentang layanan pembuatan website dan aplikasi beserta formulir kontak untuk memesan layanan.
URL repository	https://github.com/hmzahiqlball/proyek3-modul2.git
URL deployment (jika ada)	

E2. Rencana Fitur dan Acceptance Criteria

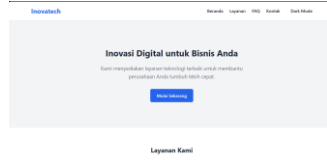
Fitur Minimal	Perilaku atau State yang Direncanakan	Acceptance Criteria Mahasiswa	Bukti
Navigasi mobile	Saat hamburger ditekan, menu muncul.	Status terbuka dan tertutup (aria-expanded) berubah. Menu tampil sesuai interaksi.	
Daftar fitur atau layanan dinamis	Fitur dirender dari JavaScript array.	Jumlah kartu sesuai dengan isi array dan tidak duplikat.	
Filter daftar	Klik salah satu filter untuk memfilter fitur.	Daftar layanan tersaring sesuai kategori. Jika kosong, muncul empty state.	
FAQ accordion	Klik pertanyaan untuk membuka/menutup.	Hanya 1 yang bisa terbuka di waktu yang sama, aksesibilitas keyboard jalan.	

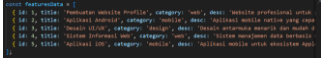
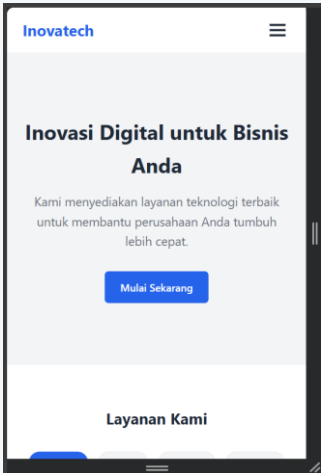
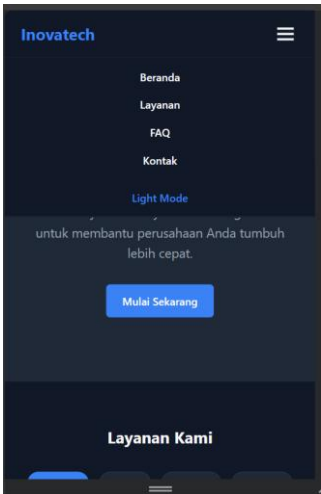
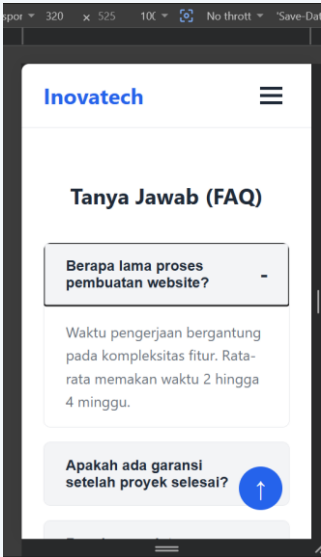
Fitur Minimal	Perilaku atau State yang Direncanakan	Acceptance Criteria Mahasiswa	Bukti
Form kontak beserta validasi	Validasi untuk nama, email, pesan kosong.	Pesan sukses bila valid, pesan error merah bila kosong/salah.	
Fitur kembali ke atas atau interaksi lain yang relevan	Scroll ke bawah, tombol muncul.	Tombol back to top memunculkan diri pada scroll tertentu dan membawa layar ke puncak saat diklik.	
Tema atau perubahan visual berbasis class	Toggle antara light-theme dan dark-theme.	Warna latar, font, dan card menyesuaikan tema gelap; teks tetap kontras.	

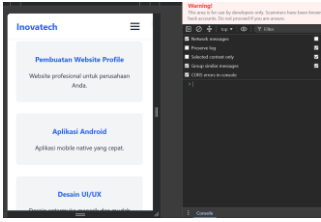
E3. Pemetaan Event, Fungsi Penanganan, dan Perubahan DOM

Fitur	Event	Fungsi Penanganan	Elemen DOM atau Class yang Berubah	State
Navigasi mobile	click pada .mobile-nav-toggle	(Anonymous function) / Arrow function	Class .active pada #main-nav, atribut aria-expanded	Buka/Tutup
Filter	click pada .filter-btn	renderFeatures(filter)	Penambahan/Penghapusan elemen dalam #features-container dan class active pada tombol filter	Filter Aktif/All
FAQ	click / keydown pada .accordion-header	(Anonymous function) / Arrow function	Gaya sebaris maxHeight pada isi akordeon, aria-expanded pada header	Expanded/Collapsed
Form	submit pada #contact-form	(Anonymous function) / Arrow function	textContent pada elemen #...-error, class .hidden pada #form-success	Valid/Invalid/Success
Tema atau state	click pada #theme-toggle	(Anonymous function) / Arrow function	Class .dark-theme pada tag <body>, konten logo tombol	Tema Terang/Gelap

E4. Riwayat Tahapan

Tahap	Tanggal	Keluaran atau Perubahan	Bukti atau Commit
Audit Modul 1	18/09/2026	Pembuatan layout HTML/CSS dasar yang bersih dan statis.	
Acceptance criteria	18/09/2026	Penentuan 7 fitur interaktif (Nav, Filter, FAQ, dsb) dan AC-nya.	Terdapat pada tabel E2.

Tahap	Tanggal	Keluaran atau Perubahan	Bukti atau Commit
Selector dan data awal	18/09/2026	Persiapan struktur JSON array di main.js (featuresData) dan variabel selector DOM.	
Fitur 1 sampai 3	18/09/2026	Mobile Nav, Render Fitur Dinamis, dan Filter terselesaikan.	
Fitur 4 sampai 7	18/09/2026	FAQ Accordion, Validasi Form Kontak, Back to Top, Dark Theme.	
Pengujian keyboard dan tampilan responsif	18/09/2026	Form dan FAQ dapat dites menggunakan keyboard Tab/Enter, tampilan aman hingga layar 320px.	

Tahap	Tanggal	Keluaran atau Perubahan	Bukti atau Commit
Perbaikan dan refaktorisasi	18/09/2026	Penghapusan console log sementara, merapikan struktur komentar di kode.	
Code selesai	18/09/2026	Seluruh proyek disimpan dan file siap dikumpulkan/di-push.	

E5. Acceptance Test Mini Project

ID	Kriteria	Status	Bukti dan Catatan
MP-01	Navigasi mobile: button dapat membuka dan menutup menu serta memperbarui aria-expanded secara sesuai.	☒ Lulus	Tombol hamburger berfungsi baik dan aria-expanded toggle true/false.
MP-02	Daftar dinamis: jumlah kartu mengikuti data dan tidak duplikat	☒ Lulus	Render bekerja menghapus cache DOM lama, lalu menambahkan node baru (createElement).
MP-03	Filter: subset tepat dan kondisi kosong ditangani	☒ Lulus	Saat array filter tidak menemukan hasil, DOM memunculkan .empty-state.
MP-04	FAQ: jawaban terkait terbuka, lainnya tertutup, keyboard bekerja	☒ Lulus	Hanya satu FAQ yang akan terbuka; ditekan Enter pada keyboard tetap jalan karena keydown event listener.
MP-05	Form: input yang tidak valid ditolak, sedangkan input yang valid menampilkan ringkasan atau pesan keberhasilan.	☒ Lulus	Event e.preventDefault() berjalan. Pesan merah muncul saat error, hijau saat sukses.

ID	Kriteria	Status	Bukti dan Catatan
MP-06	Tema atau state: class berubah dan kontras teks tetap terbaca.	☒ Lulus	<body> diberi class dark-theme via classList.toggle, kontras warna sangat jelas memakai CSS Variables.
MP-07	Tidak ada inline onclick dan tidak memakai innerHTML untuk input pengguna	☒ Lulus	Semua menggunakan addEventListener dan iterasi render JS menggunakan .textContent (aman dari XSS).
MP-08	Tidak ada error Console pada alur normal	☒ Lulus	File js dimuat dengan defer dan sudah diuji di console browser: bersih dari error.
MP-09	Viewport 320, 768, dan 1024 px tidak merusak tampilan Modul 1	☒ Lulus	Fitur CSS Grid/Flexbox dipasang, navigasi beradaptasi di <=768px.
MP-10	Fitur utama dapat digunakan dengan mouse dan keyboard	☒ Lulus	Fitur navigasi Tab berfungsi, Accordion FAQ merespon tombol.
MP-11	Folder proyek, worksheet, README, dan bukti uji lengkap	☒ Lulus	File telah tertata rapi (index.html, css/, js/).
MP-12	Saya dapat menjelaskan alur event sampai perubahan DOM	☒ Lulus	Saya memahami konsep querySelector, addEventListener, event object, dan modifikasi elemen

F. Kuesioner Self-Assessment

Skala: 1 belum mampu; 2 mampu dengan banyak bantuan; 3 mampu dengan panduan; 4 mandiri; 5 mampu menjelaskan kepada orang lain.

No.	Pernyataan	1	2	3	4	5
1	Saya dapat menggunakan tipe data, kondisi, dan function.	☐	☐	☐	☐	☒
2	Saya dapat mengolah array of objects.	☐	☐	☐	☐	☒
3	Saya dapat memilih dan mengubah elemen DOM.	☐	☐	☐	☐	☒
4	Saya dapat memasang event listener dan menjelaskan alurnya.	☐	☐	☐	☒	☐
5	Saya dapat memvalidasi form dan menampilkan pesan.	☐	☐	☐	☐	☒
6	Saya dapat menjelaskan sinkron dan asinkron.	☐	☐	☐	☒	☐
7	Saya dapat menggunakan Promise dan async/await.	☐	☐	☐	☒	☐
8	Saya dapat memakai fetch dan memeriksa response.ok.	☐	☐	☐	☐	☒
9	Saya dapat menangani loading, success, empty, dan error.	☐	☐	☐	☐	☒
10	Saya dapat membaca serta memperbaiki error JavaScript.	☐	☐	☐	☒	☐
11	Saya dapat menjelaskan seluruh kode yang dikumpulkan.	☐	☐	☐	☒	☐

Bagian paling menantang pada Modul 2:

Bagian yang paling menantang adalah mengelola urutan eksekusi pada Asynchronous JavaScript (Promise, async/await, dan fetch). Memahami bahwa proses asynchronous masuk ke dalam antrean (microtask queue) dan tidak mengeksekusi baris secara berurutan seperti kode sinkron membutuhkan penyesuaian logika, terutama saat harus mengelola perubahan state UI seperti loading, success, empty, dan penanganan error.

Konsep yang sekarang paling siap saya jelaskan kepada orang lain:

Konsep manipulasi DOM berbasis objek dan pemisahan logika. Saya sekarang paham dan siap menjelaskan bagaimana cara mengambil data array of objects, merendernya secara dinamis menggunakan kombinasi document.createElement, .textContent, dan .appendChild atau .replaceChildren() dengan aman untuk menghindari injeksi HTML secara langsung.

G. Peer Assessment

Nama Penilai	Nama Pemilik Mini Project
M. Rafi Ardiansyah	Putra Suyapratama

No.	Kriteria	Skor 1 sampai 5	Catatan Kelebihan dan Saran Perbaikan
1	Kesesuaian fitur: seluruh fitur minimal tersedia	5	Seluruh fitur seperti toggle nav, FAQ, form, dan tema sudah bekerja.

No.	Kriteria	Skor 1 sampai 5	Catatan Kelebihan dan Saran Perbaikan
2	Daftar dinamis: data, render, filter, kondisi kosong bekerja	5	Data dirender dengan baik, filter bekerja cepat, state empty muncul jika data kosong.
3	Interaksi: state konsisten dan dapat diulang	5	Toggle tema terang/gelap serta animasi Accordion berjalan secara konsisten berulang kali.
4	Validasi: input salah ditangani dan pesan membantu	5	Validasi form berjalan baik, menolak nilai kosong dan memberi teks peringatan yang jelas.
5	Keyboard: navigasi, FAQ, dan form dapat digunakan tanpa mouse	5	Seluruh komponen actionable bisa dijangkau dengan tombol Tab dan diaktivasi menggunakan Enter/Space.
6	Stabilitas: tidak ada error atau duplikasi elemen	5	Console bersih dari error/warning, dan layout aman merespon ukuran layar terkecil (320px).

Satu umpan balik dari rekan yang akan saya terapkan:

Pertimbangkan untuk menambahkan fungsi debounce atau penundaan saat memanipulasi event pencarian/filter di masa mendatang agar performa browser tidak tersendat ketika array datanya sudah sangat besar.

H. Log Bantuan dan Penggunaan AI

Prinsip AI boleh membantu menjelaskan konsep atau memberi hint setelah ada percobaan. Jawaban AI harus diverifikasi melalui dokumentasi, DevTools, Network, atau eksperimen mandiri.

Tanggal atau Fase	Pertanyaan atau Sumber	Intisari Bantuan	Hasil Verifikasi	Penyesuaian atau Perubahan yang Saya Lakukan

Tanggal atau Fase	Pertanyaan atau Sumber	Intisari Bantuan	Hasil Verifikasi	Penyesuaian atau Perubahan yang Saya Lakukan

Jika tidak menggunakan AI, jelaskan strategi debugging dan dokumentasi yang digunakan:

Strategi debugging utama saya adalah mengandalkan Developer Tools bawaan browser (Opera GX DevTools):

1. **Console:** Digunakan secara aktif dengan `console.log()` untuk melacak nilai variabel (seperti state atau data hasil iterasi) secara bertahap setiap kali merangkai fungsi baru.
2. **Network Tab:** Digunakan saat uji coba asynchronous untuk memastikan request HTTP berstatus 200 (OK) dan memvalidasi tipe isi Response JSON.
3. **Elements Tab:** Digunakan untuk inspeksi langsung guna memastikan bahwa interaksi klik benar-benar menyuntikkan class CSS (misalnya `.active`) dan atribut aksesibilitas (`aria-expanded`) ke dalam DOM secara real-time.

Untuk dokumentasi, saya secara mandiri merujuk pada dokumentasi resmi MDN Web Docs untuk memahami metode Vanilla JavaScript tertentu (seperti `document.createElement`, manipulasi class, dan konsep event delegation). Saya juga menggunakan Modul PDF Praktikum sebagai acuan utama langkah-per-langkah pengerjaan (Acceptance Criteria).

H1. Bukti Pemahaman

Pilih satu saran yang Anda terima. Jelaskan kembali konsepnya dengan bahasa sendiri:

Saya akan menerapkan konsep **debounce** pada event interaksi yang terjadi dengan cepat. Konsep ini berarti kita "menunda" pemanggilan fungsi eksekusi (seperti `render`) sampai pengguna selesai memberikan aksi selama jeda waktu tertentu (misalnya 300 milidetik). Jika aksi terjadi sebelum jeda habis, waktu tunggu akan direset. Hal ini mencegah fungsi yang berat berjalan ratusan kali secara tak perlu, sehingga menghemat memori (resource).

Eksperimen yang membuktikan saran tersebut benar atau tidak:

Saya dapat membuktikannya dengan membuat event listener pada input teks dan meletakkan `console.count('Render Filter')` di dalam fungsinya.

1. **Tanpa debounce:** Saat saya mengetik 10 karakter secara cepat, teks "Render Filter" muncul 10 kali di Console.
2. **Dengan debounce (menggunakan `setTimeout` & `clearTimeout`):** Saat saya mengetik 10 karakter secara cepat, "Render Filter" hanya dieksekusi 1 kali saat saya berhenti mengetik. Ini membuktikan kinerja browser jadi jauh lebih hemat dan efisien.

H2. Batas yang Saya Patuhi

- ☒ Saya menulis prediksi dan percobaan awal sendiri.
- ☒ AI tidak menghasilkan seluruh Task 1, Task 2, Task 3, homework, atau mini project.
- ☒ Pada debugging, saya membaca error, baris, selector, atau nilai aktual sebelum meminta hint.
- ☒ Prompt dan dampaknya dicatat ketika AI digunakan.
- ☒ Saya dapat menjelaskan dan mengubah kode tanpa AI saat coding.

I. Lesson Learnt

Konsep JavaScript: Konsep JavaScript apa yang sekarang dapat Anda jelaskan dengan contoh?

Konsep manipulasi DOM menggunakan sintaks Vanilla JavaScript. Contohnya, menggunakan `document.createElement('div')` untuk membuat elemen baru, `element.textContent = 'Halo'` untuk mengisi teks secara aman (tanpa Cross-Site Scripting yang rentan terjadi jika memakai `.innerHTML`), dan `container.appendChild(element)` untuk merendernya ke dalam layar.

Sinkron dan Asinkron: Apa perbedaan terbesar yang Anda rasakan antara kode sinkron dan asinkron?

Kode sinkron mengeksekusi perintah secara berurutan dan akan memblokir baris kode di bawahnya hingga baris saat ini selesai. Sedangkan kode asinkron (seperti `fetch` atau `setTimeout`) akan dieksekusi di background dan membiarkan baris kode berikutnya tetap berjalan. Hal ini sangat terasa ketika mengambil data JSON, di mana kita harus menggunakan blok `.then()` atau `await` agar UI tidak macet sambil menunggu proses selesai.

Kesalahan yang Memberikan Pembelajaran: Error `fetch` atau JSON apa yang paling membantu pemahaman?

Error HTTP 404 (Not Found) pada saat melakukan `fetch`. Awalnya saya berpikir `catch` akan langsung menangkap error tersebut, namun ternyata `fetch` hanya melempar error ke `catch` jika koneksi jaringannya putus. Dari situ saya belajar bahwa saya harus secara eksplisit mengecek status `response.ok` dan melempar Error (`throw new Error`) secara manual jika statusnya bukan 200.

Keputusan Teknis: Keputusan teknis apa yang Anda ambil dan apa alasannya?

Keputusan untuk tidak menggunakan `innerHTML` ketika merender masukan teks bebas dari pengguna dan menggunakan class-based toggle (seperti `classList.toggle('dark-theme')`) di CSS alih-alih memanipulasi inline styles (`style.backgroundColor`) secara langsung lewat JS. Alasannya adalah pemisahan peran yang lebih rapi (JS untuk logika logika dan CSS murni untuk tampilan visual), serta memastikan keamanan eksekusi dari input palsu.

Kemandirian: Bagian mana yang diselesaikan tanpa bantuan?

Perancangan layout HTML dan styling CSS pada tahap awal (Modul 1 dan konversi statis), serta pengaturan alur logika percabangan untuk penanganan filter layanan.

Umpan Balik Rekan: Umpan balik peer apa yang anda terapkan?

Saran untuk menambahkan fungsi `debounce` pada filter interaktif untuk mengoptimalkan kinerja render (meringankan kerja memori browser) saat mengubah daftar layanan yang besar.

Target Berikutnya: Kompetensi apa yang menjadi target berikutnya?

Menguasai integrasi Vanilla JavaScript dengan framework modern (seperti `React.js` atau `Vue`) untuk melihat seberapa banyak kode DOM yang bisa dipangkas dan diotomatisasi (state-driven UI).

J. Checklist Akhir Modul 2

J1. Kualitas Kode dan Interaksi

- ☒ File JavaScript dipanggil dengan `defer` dan tidak menghasilkan error saat halaman dibuka.
- ☒ Nama variabel dan function menjelaskan isi atau tindakan.
- ☒ Input dikonversi dan divalidasi sebelum digunakan.
- ☒ Function logika dapat diuji terpisah dari DOM.
- ☒ Selector sesuai dengan HTML dan tidak ada `null` yang tidak dijelaskan.
- ☒ Interaksi menggunakan `addEventListener`, bukan atribut event inline.
- ☒ Perubahan visual memakai class CSS.

- ☑ Button dan form dapat digunakan dengan keyboard.
- ☑ Pesan error spesifik dan tidak hanya mengandalkan warna.
- ☑ Proses asynchronous memiliki state idle, loading, success, empty, dan error.
- ☑ fetch memeriksa response.ok dan menangani kegagalan HTTP atau parsing.
- ☑ Tidak ada pesan error yang belum dijelaskan pada Console.

J2. Kelengkapan Pengumpulan

Komponen	Status	Lokasi atau Nama File
Worksheet PDF	☑	251511056_PutraSuyapratama_Worksheet_Modul2.pdf (Di folder root Tugas)
Task 1 beserta bukti pengujian	☑	Source-code/praktikum/praktikum-task-1 /
Task 2 beserta tangkapan layar skenario pengujian	☑	Source-code/praktikum/praktikum-task-2/
Task 3 beserta bukti dari Console dan Network	☑	Source-code/praktikum/praktikum-task-3/
Homework beserta minimal tujuh tangkapan layar pengujian	☑	Source-code/homework/
Mini project beserta README	☑	Source-code/miniproject/
Log bantuan dan penggunaan AI	-	
Repository atau deployment dapat diakses	☑	

Catatan akhir sebelum mengumpulkan:

Seluruh tugas telah selesai dikerjakan sesuai batas ketentuan teknis (menggunakan murni Vanilla JavaScript, tidak memakai framework UI, mematuhi standar aksesibilitas, tanpa innerHTML yang berbahaya, dan tanpa inline event listener). Seluruh pengerjaan juga berjalan lancar tanpa terdeteksi error (merah) di jendela Console. Terima kasih.